Análise de arquivo suspeito

Share

exec.pyc

PYC

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Avaliando se a análise é defensiva

Vou começar lendo as instruções de leitura de arquivos e depois inspecionar o arquivo de forma segura, apenas com análise estática. Ele não será executado em nenhum momento.

Ler o skill de leitura de arquivos antes de inspecionar o upload SKILL.md

Não há uma seção específica para `.pyc`, então vou seguir a rota genérica: identificar o arquivo, ver os bytes iniciais e calcular os hashes. Tudo isso é somente leitura.

Ran 7 commands

Já tenho elementos suficientes para o veredicto. Para corroborar, vou verificar se esses indicadores (o endereço de carteira e o hash) aparecem em relatórios públicos de ameaças.

Searched the web

Veredicto: é malware. O arquivo é um *loader* (carregador de primeiro estágio) ofuscado, que coincide com uma campanha documentada publicamente. A análise foi 100% estática: não executei o arquivo nem contatei nenhum servidor.

Passo a passo

1. **Identificação.** `exec.pyc` tem 7.001 bytes e é bytecode compilado para Python 3.13 (magic number `f30d0d0a`). O SHA-256 é `32aa84af…775e`. O nome do fonte original embutido é um hash de 64 caracteres, não um nome de módulo real.
2. **Estrutura do módulo.** Carreguei só o objeto de código, sem importá-lo. As variáveis têm nomes aleatórios de 15 letras. O módulo se resume a `__import__`, `b64decode`, `decompress`, `compile` e `exec` aplicados a um bloco de 6.096 caracteres, ou seja, ele decodifica algo escondido e executa. Ofuscação sozinha não prova nada, pois ferramentas legítimas também fazem isso. Por isso fui ver o que havia dentro.
3. **Abrindo as camadas sem executar.** O bloco passa por base64, depois zlib, depois XOR com a chave 134. O resultado são cerca de 15,5 KB de código Python legível (438 linhas), que examinei só como texto e árvore sintática (AST).
4. **O que o código interno faz:**

- **Evita alvos russos.** A função `_isRussianSystem` consulta as variáveis de idioma (`LANG`, `LANGUAGE`, `LC_ALL`) e o fuso horário do Windows (`tzutil`). Tudo indica que o programa aborta nesse caso, um comportamento típico de malware.
- **C2 na blockchain.** Consulta RPCs públicos da Solana para ler as transações de uma carteira fixa e extrair delas um link em base64 para a próxima etapa. Assim não há servidor fixo para derrubar.
- **Baixa e executa código remoto de forma oculta.** Baixa o Node.js v22.9.0 de nodejs.org para a pasta do usuário e obtém o JavaScript da etapa seguinte. Os cabeçalhos `ivbase64` e `secretkey` indicam que esse conteúdo vem cifrado. O código o grava como `i.js` e o executa **sem janela e com a saída descartada.**
- **Controle de execução.** Grava `~/init.json` com timestamps.

5. **Corroboração pública.** A carteira Solana embutida no código é a que a StepSecurity documenta como C2 da campanha ForceMemo, e é a mesma usada pelo GlassWorm, um worm que atinge extensões do VS Code e do OpenVSX desde outubro de 2025. No ForceMemo, centenas de repositórios Python no GitHub foram adulterados por contas roubadas e force-push. Os relatórios listam como indicadores o download do Node.js v22.9.0 e o arquivo ~/init.json, além de uma variável marcadora chamada lzcdrtfxyqiplpd, que é exatamente um dos nomes aleatórios deste arquivo.
ForceMemo: Hundreds … +3

Conclusão

Ofuscação em camadas, checagem geográfica, C2 em blockchain e execução oculta de um runtime baixado não deixam explicação benigna plausível, e os indicadores batem com os publicados. Minha confiança no veredicto é muito alta.

O arquivo é só a porta de entrada. O JavaScript final é entregue sob demanda, e a carteira acumula dezenas de transações de memo com URLs de C2 que mudam ao longo do tempo. Por isso não dá para saber, só por este arquivo, o que o payload faria na sua máquina. breakglass

Se ele chegou a ser executado

- **Procure os artefatos.** Verifique `~/init.json`, a pasta `~/node-v22.9.0-*` (e o `.zip` ou `.tar.*` correspondente), um `i.js` ao lado do script Python que o rodou, e um processo `node` sem janela.
- **Assuma comprometimento.** A partir de **outro dispositivo limpo**, troque senhas e revogue tokens e chaves (GitHub, npm, PyPI, nuvem, SSH). Se havia carteiras de criptomoeda na máquina, mova os fundos. Reinstale o sistema se não conseguir confirmar que está limpo.
- **Se veio de um repositório,** considere-o comprometido e avise o mantenedor.
- **Atenção na detecção.** O tráfego vai para nodejs.org e RPCs públicos da Solana, domínios legítimos que filtros por reputação não barram. Vale procurar o comportamento: Python iniciando `node` a partir da pasta do usuário.

Indicadores (IOCs)

Tipo	Valor
SHA-256	`32aa84af35da78e1e8707e7d93fd48873e30d4f559d514a2648c07e43d35775e`
Carteira Solana (C2)	`BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC`
Variável marcadora	`lzcdrtfxyqiplpd`
Artefatos no disco	`~/init.json`, `~/node-v22.9.0-*`, `i.js`
Fonte embutido	`566bb30ad42762b4584c55267158ea901602fc149fb2e0911107e49432de1249.py`

Não revi nem reproduzi o payload final por escolha: o necessário para o veredicto e para a defesa está acima. Se quiser, posso ajudar a verificar um repositório suspeito ou a montar um checklist de resposta ao incidente.

Qual é a categoria principal deste malware?

Confirmando as técnicas ATT&CK e a menção a roubo de dados ▸

Categoria principal: downloader/loader de primeiro estágio, o que muitos antivírus rotulam como *Trojan-Downloader*.

O arquivo não rouba dados, não criptografa nada e não abre acesso remoto por conta própria. Ele só prepara o terreno para outra coisa chegar. Esse é o ciclo típico de um downloader:

1. Esconde-se atrás de camadas de ofuscação.
2. Decide se deve rodar, evitando sistemas russos.
3. Descobre onde está a próxima etapa, lendo a carteira Solana.
4. Baixa um runtime (Node.js) e o JavaScript da etapa seguinte.
5. Executa tudo de forma oculta.

Tecnicamente é um *downloader* e não um *dropper*, porque o payload não vem embutido. Ele é buscado remotamente, e as camadas base64/zlib/XOR são só empacotamento em volta da lógica de busca.

Isso não o torna inofensivo. Um loader dá ao atacante a capacidade de entregar o que quiser depois e de trocar o payload sem alterar este arquivo. Sobre a etapa seguinte, a única informação segura é a de que ela não está no arquivo e que não dá para saber, só por ele, o que faria. Quanto à distribuição, a campanha ForceMemo espalhou esse tipo de

código por centenas de repositórios Python do GitHub adulterados, ou seja, o vetor é de cadeia de suprimentos (*supply chain*). stepsecurity

Em termos de MITRE ATT&CK, o comportamento observado mapeia para:

Técnica	O que foi observado
T1027, Obfuscated Files or Information	Camadas base64, zlib e XOR
T1614.001, System Language Discovery	Checagem de idioma e fuso para evitar alvos russos
T1102.001, Dead Drop Resolver	Link da próxima etapa lido na blockchain da Solana
T1105, Ingress Tool Transfer	Download do Node.js e do JavaScript
T1059.006 e T1059.007, Python e JavaScript	Execução do `i.js` pelo Node baixado
T1564.003, Hidden Window	Execução sem janela e com a saída descartada

Create a spreadsheet